

Flexiple Engineering Challenge: The Sourcing Refinement Loop

Candidate assignment - build one small, real slice of Flexiple's AI recruiter.

Context

Flexiple is building an AI recruiter. Its first job is sourcing: we maintain a talent map of roughly 98 million people, and a recruiter starts by typing what they are looking for in free text, the way they would type a Google search. For example: "RDS developers with 4-7 years of experience who have worked at startups, for a role based in Bangalore."

From that one sentence, the product generates two things: a set of structured, objective filters (skills, years of experience, location, company background) and a subjective fit rubric that captures what good looks like for this specific role. It applies the objective filters to the talent map, then uses an LLM to score the remaining profiles against the rubric.

The part that makes it useful is the refinement loop. The product shows the recruiter a few profiles at a time and asks, in effect, "does this match?" From the recruiter's yes/no feedback, it keeps tweaking the filters and the rubric until the recruiter is happy and freezes the search.

Your task is to build this loop as one small, focused, delightful product.

Time box

You have 3 hours in total:

- 2.5 hours to build
- 30 minutes to review your work, write the README, and record your Loom walkthrough (max 15 minutes)

Please stop after 3 hours. We care far more about your decisions, prioritisation, and quality of execution than about how many features you complete.

The challenge

Build a full-stack application that runs the sourcing refinement loop end to end for a single

search session:

1. Free text to filters and rubric. The recruiter types a free-text requirement. Your server calls a real LLM API to generate (a) structured objective filters and (b) a subjective fit rubric. The recruiter can see both.
2. Filter. Apply the objective filters to the supplied sample dataset (see Sample data). This runs locally against the file; there is no real 98M-person database in this exercise.
3. Score and rank. Use the LLM to score the filtered profiles against the fit rubric and rank them.
4. Refine in conversation. Show the recruiter 4-5 profiles at a time. The recruiter tells you in chat which profiles match and which do not. Your app uses the LLM to adjust the filters and rubric based on that feedback, re-runs the search, and shows the updated results. This repeats as many times as the recruiter wants.
5. Freeze. When the recruiter freezes the search, show the final filters, the final rubric, and the final ranked list of matching profiles.

The experience we want you to build

We are deliberately specifying the user flow so you can spend your time on quality, not on guessing what to build:

1. The recruiter lands on a clean search screen with a single free-text input.
2. On submit, the app shows that it is thinking, then presents the generated filters and rubric in a format the recruiter can read at a glance and edit directly.
3. The app runs the search and shows the top 4-5 profiles, each with a short, specific explanation of why it matched, tied to real details in that profile rather than generic praise.
4. The recruiter reacts in chat - "1 is too junior, 2 and 4 are right" - or with per-profile yes/no. The app says what it changed in the filters or rubric and why, re-runs the search, and shows the new set. The current filters and rubric are always visible.
5. The recruiter freezes the search and sees a clear final state: the frozen filters, the frozen rubric, and the ranked shortlist.

Every state should be designed: first load, thinking moments, empty results, LLM failures, and the frozen summary. The frontend is half of this assignment, not a shell around the backend. We want an experience a recruiter would choose to use again.

The LLM requirement

- Real, server-side LLM API calls only. No mocked or canned LLM responses. Any model with a free tier is fine (for example Google Gemini, Groq, or the free models on OpenRouter). Read the API key from an environment variable and never commit it.
- Keep your prompts in the repository where we can read them. They are part of what we evaluate.
- Use structured output for anything you render or apply (filters, rubric, scores), and validate it. Malformed output, rate limits, timeouts, and slow responses should be handled deliberately rather than crash the app.
- Explanations for why a profile matched must cite actual fields from that profile.

Sample data

We supply profiles.json alongside this assignment: 48 fictional candidate profiles as a JSON array. Load it locally and treat it as the entire talent pool. Each record has this shape:

```
{
  "id": "p01",
  "name": "Ananya Rao",
  "current_title": "Senior Backend Engineer",
  "years_experience": 6,
  "location": "Bangalore",
  "current_company": "NimbusPay",
  "current_company_type": "startup",
  "skills": ["Node.js", "PostgreSQL", "AWS RDS", "Redis", "TypeScript"],
  "past_companies": [
    {"company": "Zeta", "company_type": "scaleup", "title": "Backend Engineer", "years": 3}
  ],
  "education": "B.E. Computer Science, BITS Pilani",
  "summary": "Backend engineer who has spent most of her career building payments infrastructure at early-stage startups."
}
```

Notes on the data:

- company_type is one of: startup, scaleup, enterprise, agency.
- The dataset deliberately mixes obvious matches, near misses, and clear non-matches for typical searches, so the refinement loop has something to bite on.

Ground rules

- Use any language or framework. We do not believe language familiarity is a useful signal of engineering ability when AI tools help people move quickly across stacks. We care about the technical decisions you make, the user decisions you make, and the quality of the finished experience.
- AI coding tools are fine and expected. We evaluate what you ship and the judgment behind it.
- We should be able to run your app locally with one or two commands after setting the LLM API key environment variable.
- Stay in scope: no login, no multiple roles, no persistence across sessions, nothing outside the flow above. Spend the time on quality instead.

What to submit

- A repository we can run, with a README covering setup, the name of the API key environment variable, and a short decisions section: what you prioritised, what you cut, and why.
- Your LLM prompts, in the repository.

- A Loom walkthrough of at most 15 minutes showing the full loop: one search from free text to freeze, at least one refinement round driven by recruiter feedback, and one failure or recovery moment (for example a rate limit or a malformed response handled gracefully).

What we are evaluating

- Whether the loop works end to end with real LLM calls.
- The quality of your technical decisions: how you structure the LLM interactions, how you carry state across refinement rounds, and how you handle failure.
- The quality of your user decisions: hierarchy, clarity of the filters and rubric, trust in the explanations, and how designed the loading, empty, and error states feel.
- Whether refinement visibly responds to feedback in a way a recruiter would trust.